

UrbanLens — Technical Reference & Judge Preparation

Every library, every algorithm, why it was chosen, and how to defend it. Written for the team to study from.
SIH 2026 · PS-SW-001 · commit **d54406e**.

How to use this document

Sections 1–8 are the material — read your module in depth and skim the rest, because judges ask across boundaries. Section 9 is the module ownership map. Section 10 is a question bank with answers. Section 11 is the list of weaknesses and how to answer them **honestly** — rehearse these hardest, because a confident, specific admission of a limit reads as competence, while a bluff that unravels costs the whole demo.

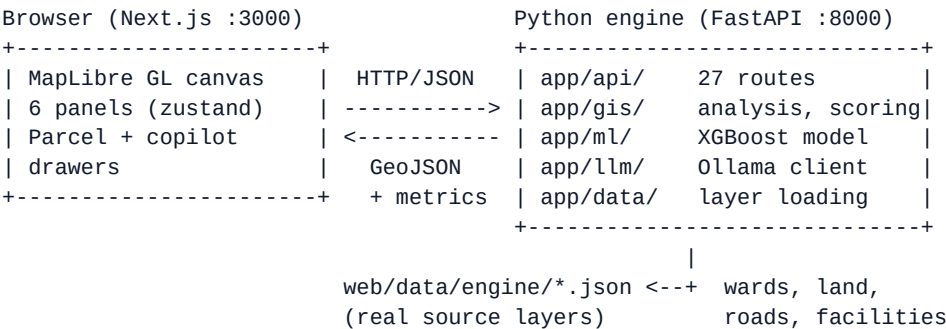
1. What UrbanLens is, in one paragraph

GLIS tells a planner what land exists. UrbanLens tells them what is happening on it, what is likely next, what is missing, and what should be built where — then simulates the decision before it is made. It is a spatial decision-support system, not a dashboard: every figure is computed by a Python GIS engine from real boundaries, real OpenStreetMap infrastructure and census-grounded population, and every recommendation explains itself.

The one-line pitch: *UrbanLens turns land records, infrastructure and population into an explainable answer to three questions — what should we build, where, and why?*

2. Architecture

Two processes. This split is the single most important thing to be able to draw on a whiteboard.



The rule that governs the split: the browser renders and the engine computes. The UI holds no analysis of its own. Two exceptions, both deliberate and neither re-deriving anything: the suitability weight sliders re-blend factor scores the engine already returned, and land-use transitions pivot the parcel layer already in memory.

3. The stack — what, why, and what we rejected

Frontend

Library	Version	What it does	Why this one
Next.js	14.2.15	React framework, routing, build	App Router + static export; the brief names it (§39)
React	18.3	UI components	Named in the brief
TypeScript	5.6	Static types	GeoJSON shapes are easy to get wrong; the compiler catches it
Tailwind CSS	3.4	Utility styling	Named in the brief; fast iteration on a dense dashboard

Library	Version	What it does	Why this one
Radix UI	1.x	Slider, Switch, Tooltip	Accessible primitives — keyboard and ARIA handled. shadcn/ui is built on Radix (§39)
MapLibre GL JS	4.7	Vector map rendering	Named in the brief (§40). Open-source fork of Mapbox GL — no token, no licence fee, WebGL vector tiles
zustand	4.5	State management	Chosen over Context so the always-mounted map canvas does not re-render the React tree on every hover
framer-motion	11.18	Animation	Declarative transitions; MotionConfig gives app-wide reduced-motion in one line
Recharts	2.13	Charts	React-native chart composition for the built-up trajectory
three.js / R3F	0.185	Landing globe	The 3D earth on the landing page only — not used in the product
Turf.js	7.4	Client-side geometry	Small helpers only; the real geometry is server-side

Backend

Library	Version	What it does	Why this one
FastAPI	0.135	HTTP API	Named in the brief (§41). Async, automatic OpenAPI docs at /docs, Pydantic validation built in
Pydantic	2.12	Request validation	Typed request bodies; rejects bad input before it reaches the engine
uvicorn	0.44	ASGI server	Standard FastAPI runner
Shapely	2.1	Geometry ops	Contains, centroid, union, STRtree index. Version 2.x is vectorised over numpy
pyproj	3.7	Projections	Coordinate transforms (EPSG:4326 <-> metric)
NumPy	2.4	Arrays / raster maths	The population grid is a numpy array; every window query is vectorised
pandas	3.0	Tabular handling	Census and training-frame assembly
scikit-learn	1.8	Split, CV, metrics	train_test_split, cross_val_score, roc_auc_score, accuracy_score
XGBoost	3.4.1	The growth model	Gradient-boosted trees — see §6

Deliberately not used — be ready for this

Not used	Brief says	Why not, and the honest answer
PostGIS	“mandatory” (§42)	No database at all. At 2,567 parcels the city fits in memory and every query is sub-second. Every ST_* function listed has a direct equivalent here (see §5). It becomes necessary past one city
GeoPandas / GDAL	listed (§43)	Both need system GDAL, which breaks installs on teammates' machines. Shapely + pyproj cover every operation used
SQLAlchemy	listed (§41)	There is no database to map
deck.gl	listed (§40)	For very large datasets. MapLibre handles this volume comfortably
CesiumJS	optional (§40)	The brief explicitly says do not make 3D mandatory and that 2D GIS matters more
OSRM / GraphHopper	preferred (§45)	The brief permits distance approximations for the prototype. We use straight-line haversine at fixed walk/drive speeds
Deep learning	“only if time” (§11)	The brief prioritises explainability. Tree ensembles give feature importances a planner can read

4. The data — what is real and what is modelled

Being precise here is the difference between credibility and embarrassment. Every layer carries its own provenance, served on `/api/health` and `/api/layers`. The weakest layer in play is what the UI reports.

Layer	Provenance	Detail
Ward boundaries	Official	Digitised municipal ward maps — 48 AMC wards, 11 GMC wards. Real geometry, area, perimeter, compactness
Land parcels	Real (OSM)	2,567 mapped land polygons with their real land-use tag. Surveyed blocks and estates, not cadastral title plots — GLIS is not public
Facilities	Real (OSM)	1,252 across 10 types via the Overpass API, de-duplicated and re-classified
Roads	Real (OSM)	2,480 major segments
Population	Derived	Census 2011 AMC total (5,570,585) projected to 2026 at ~2.3%/yr, distributed across wards by $\text{area} \times \text{road-density}^{1.35}$
Ownership	Modelled	No public dataset records tenure. OSM confirms public ownership for 10 of 2,567; the rest is modelled deterministically from the OSM id
Official zoning	Modelled	Development-plan sheets are not machine-readable. Generated as concentric rings (commercial core -> residential -> agricultural fringe) so conflicts are meaningful
Built-up history	Modelled	2018 / 2022 / 2026 — not observed from imagery
Growth prediction	Modelled (ML)	XGBoost on real OSM land-use labels

Two layers currently overstate themselves

“Vegetation & NDVI Canopy” and “Urban Heat Island (UHI)” are computed from a radial distance formula, not from satellite bands. NDVI and UHI are specific, well-known satellite products. If a judge asks how NDVI was computed, the honest answer contradicts the label. **Fix before demo:** remove them, or relabel “illustrative — not satellite-derived”. Do not defend them as real.

5. Spatial methods — the GIS you must be able to explain

STRtree — the spatial index

A Sort-Tile-Recursive R-tree. Without it, “which ward contains this point?” means testing the point against all 48 ward polygons, each with hundreds of vertices. The tree stores bounding boxes hierarchically so a query descends to a handful of candidates in $O(\log n)$, then only those get the exact geometric test. **This is the same structure PostGIS builds for a GiST index** — a useful thing to say when asked about the database.

Prepared geometry

`shapely.prepared.prep()` pre-computes an internal representation of a polygon so repeated point-in-polygon tests are much faster. Used when rasterising wards, where the same 48 polygons are tested against tens of thousands of grid cells.

Haversine distance

Great-circle distance between two lat/lng points on a sphere. Used everywhere distance is needed. It is straight-line, not road-network distance — an approximation the brief permits for a prototype (§45). Vectorised over numpy arrays so a whole grid is one operation.

$$a = \sin^2(d\text{lat}/2) + \cos(\text{lat}1) \cdot \cos(\text{lat}2) \cdot \sin^2(d\text{lng}/2)$$

$$d = 2 * R * \arcsin(\sqrt{a}) \quad R = 6371 \text{ km}$$

The population raster — and why population is conserved

Wards are rasterised once into a ~250 m grid (the WorldPop-style representation §34/§72 asks for). Each cell is assigned to the ward that contains its centre, then **each ward's census total is redistributed across only the cells it owns, in proportion to cell area**. That means summing the whole grid returns the city total exactly — rasterisation cannot lose or invent people. `npm run demo` asserts this: 7,200,002 vs 7,200,002.

Why bother: every parcel score, every ward gap and every simulation needs “how many people live within X km”. Intersecting buffers against real ward polygons hundreds of thousands of times is far too slow; an indexed sum over a small window of array cells is instant.

Cell size trade-off

250 m. Smaller means more precision and quadratically more cells; larger means coarse catchments. At 250 m Ahmedabad is 10,504 cells — trivial for numpy — and the error on a 3 km catchment is well under a percent.

6. The machine learning — the section judges probe hardest

What problem is actually being solved

Binary classification. Given a location's geometry-derived features, how likely is it that this location *looks like* already-developed land?

Element	Value
Algorithm	XGBoost gradient-boosted decision trees (XGBClassifier)
Labels	Real — OpenStreetMap's own land-use tags. 1 = residential/commercial/industrial/institutional; 0 = farmland/green/vacant/water
Samples	3,224 land polygons (Ahmedabad); 1,825 positive
Features	9, all measured from real geometry (below)
Split	75/25 stratified, random_state=42
Hyperparameters	n_estimators=300, max_depth=5, learning_rate=0.08, subsample=0.9, colsample_bytree=0.9, eval_metric=logloss
Validation	5-fold cross-validation on ROC AUC

The nine features, and their learned importance (Ahmedabad)

Feature	Meaning	Importance
built_km	Distance to nearest already-built parcel	0.325
area_ha	Parcel area in hectares	0.255
core_km	Distance to nearest urban core	0.084
pop_3km	Catchment population within 3 km	0.060
road_km	Distance to nearest arterial road	0.058
pop_density	People per km ² at this point	0.058
transit_km	Distance to nearest bus stop or metro	0.057
school_km	Distance to nearest school	0.056
hospital_km	Distance to nearest hospital	0.048

Read this out loud in the demo: the model learned that proximity to existing development dominates, which is exactly how cities actually grow — infill and edge expansion, not leapfrogging. The model was not told that; it discovered it. That is the argument for using ML here at all.

Results across all four study areas

Study area	Samples	Positives	Accuracy	ROC AUC	5-fold CV
Ahmedabad	3,224	1,825	0.880	0.956	0.939 ± 0.038
Gandhinagar	1,766	963	0.885	0.958	0.920 ± 0.030

Study area	Samples	Positives	Accuracy	ROC AUC	5-fold CV
Ahmedabad–Gandhinagar	3,884	2,113	0.883	0.955	0.940 ± 0.025
Ahmedabad Metro	4,930	2,521	0.895	0.962	0.932 ± 0.023

Leakage control — the detail that shows rigour

built_km is “distance to the nearest already-built parcel”. If a positive parcel could count *itself* as a built reference, distance would be zero and the model would simply memorise its own label. Built references are therefore taken from a **held-out half of the positives** (built_pts[:2]). Mention this if asked about overfitting — it is a concrete answer.

The honest limit — rehearse this until it is natural

This model scores **development pressure**, not a dated 2030 forecast. It answers “how developed does this location look, given roads, people and existing development?” On undeveloped land that reads as pressure — a real planning signal. A genuine “will this urbanise by 2030” model needs **observed built-up extent at two dates** (GHSL or Sentinel-2), which this repo does not hold; the 2018/2022/2026 history is modelled, so training on it would only teach the model the formula that generated it. We left the temporal model unbuilt and said so rather than dressing it up. The feature pipeline is already correct — only the label would change.

7. The scoring engine — multi-criteria decision analysis

This is what makes the product *explainable* rather than a black box, and it is the brief’s §18–20. No ML here at all — deliberately.

The formula (§18, weights exactly as the brief specifies)

$$\text{UDS} = 0.25 \cdot \text{Accessibility} + 0.20 \cdot \text{PopulationNeed} + 0.15 \cdot \text{Transit} + 0.15 \cdot \text{Infrastructure} + 0.15 \cdot \text{Environment} + 0.10 \cdot \text{LandCompatibility}$$

each factor normalised 0–100; weights renormalised by their sum, so any user-supplied weighting is valid

How a raw measurement becomes a 0–100 score

Two primitives do all of it:

```
decay_score(x, good, bad)  # 'smaller is better' – e.g. distance
    100                    if x <= good
    0                      if x >= bad
    linear                 in between
```

```
norm(x, lo, hi)            # 'bigger is better' – linear 0..100
```

Everything is deterministic. Run it twice, get the same number. No randomness anywhere in scoring — that is PRD §70’s requirement and it is worth stating plainly.

Why weighted linear, not AHP or TOPSIS

A planner has to be able to look at a slider and predict what happens. A linear weighted sum is the only common MCDA method where moving “Environment” from 15% to 30% has an effect you can explain in one sentence. AHP needs pairwise-comparison matrices a user must fill in; TOPSIS ranks by distance to an ideal solution, which is harder to justify to a committee. Explainability was the brief’s explicit priority (§11, §19).

Constraints vs weights — a design decision worth defending

“Nobody new would be served” **disqualifies** a site for a service facility; it is not a small deduction. Otherwise the ranking puts dense, already-well-served central land first — which then simulates as zero improvement, and the whole story collapses. Minimum unserved population defaults to 5,000.

8. Simulation and the copilot

How the simulator computes before/after

Sample ~340 points on a disc of radius 1.8× the facility's service radius, weighted by population density at each point. For each sample, compute distance to the nearest existing facility of that type, then to the proposed one. Coverage is the population-weighted share within the service radius; average distance is the population-weighted mean. The 1.8× window matters: sampling only inside the service radius would make “after” trivially 100%.

The copilot — and PRD §29, the rule that must not be broken

```
question -> LLM picks a tool + arguments      (language)
          -> the GIS engine runs that tool      (ALL analysis)
          -> LLM restates the result as prose   (language)
          -> raw result returned alongside prose
```

The LLM never computes a spatial result. It does two jobs: choose a tool, and phrase the output. Every number originates in the engine and is returned to the UI as structured data, so the interface renders real values rather than parsing them back out of a sentence. Seven tools: `site_search`, `explain_parcel`, `infrastructure_gaps`, `government_land`, `zoning_conflicts`, `land_use_change`, `help`.

Ollama (llama3.2) runs it locally when available. When it is not, a deterministic pattern router answers the same questions using the same tools — identical figures, fixed wording. That is not a degraded mode; it is how the demo survives a machine with no model pulled.

9. Module ownership — who studies what

Split so each person owns a vertical slice they can defend alone, and knows the two adjacent to them.

Module	Files to know	Must be able to explain
Spatial engine core	backend/app/gis/raster.py, parcels.py	STRtree, prepared geometry, haversine, the 250 m grid, population conservation
Scoring & site selection	backend/app/gis/scoring.py, analysis.py	The UDS formula, decay_score/norm, the 10 project specs, constraints vs weights, the explanation generator
Machine learning	backend/app/ml/development_model.py, prediction.py	XGBoost, the 9 features, labels from OSM tags, ROC AUC, cross-validation, leakage control, the temporal caveat
API & copilot	backend/app/api/routes.py, llm/	The 27 routes, the \$29 LLM/GIS split, the 7 tools, the deterministic fallback
Frontend & map	web/components/map/, lib/store.ts, config/layers.ts	MapLibre layers, zustand state, why the map never unmounts, the 15 layers and 6 mode presets
Data pipeline & provenance	web/scripts/, refined/, README data table	Where each layer came from, Overpass, census projection, what is modelled and why that is disclosed

10. Judge question bank

Opening / product

Q. What problem are you solving?

Raw GIS tells a planner what land exists. It does not tell them where the city is growing, which wards are underserved, which government land is suitable, or what happens if they build. UrbanLens closes that gap with a single chain: detect growth -> find the gap -> identify land -> recommend a site -> simulate impact -> explain the decision.

Q. Who is the user?

A government urban planner — AUDA or a municipal corporation. The design target is a planner preparing a development-plan recommendation, not a citizen-facing map.

Q. Why should I believe your numbers?

Because you can check them. Every layer states its own provenance on /api/health. Scores come from published deterministic formulas, not random values. Run the same query twice and you get the same answer. And where the data is thin — OSM maps only 121 schools for 7.2M people — we label the score low-confidence rather than shipping it as a finding.

Data

Q. Is this real GLIS data?

No, and we never claim it is. GLIS cadastral records are not public. We use OpenStreetMap land polygons — real mapped boundaries with their real land-use tags — which are surveyed blocks and estates rather than title plots. The architecture takes GLIS the moment it is available: it is the same parcel schema.

Q. Where does population come from?

Census 2011 municipal totals, projected to 2026 at about 2.3% a year, then distributed across wards in proportion to $\text{area} \times \text{road-density}^{1.35}$. Road density is a real measured proxy for urban intensity. These are estimates and we label them as derived, never as census ward counts.

Q. How do you know ownership is government?

Mostly we do not, and we say so. OSM confirms public ownership for 10 of 2,567 parcels. The rest is modelled deterministically, seeded per polygon from its OSM id so it never shifts between runs. It is disclosed on the health

endpoint and in the UI.

Q. Your zoning conflicts — are those real violations?

No. Official development-plan sheets are not published machine-readably, so the official designation is modelled. The conflicts demonstrate the detection method and would become real the day a DP layer is loaded. The API response says exactly this.

Machine learning

Q. Which algorithm and why?

XGBoost — gradient-boosted decision trees. Three reasons. It handles the mixed-scale, non-linear features here without scaling. It gives feature importances, which is what makes the output explainable to a planner. And at a few thousand samples it outperforms a single tree or logistic regression while training in seconds.

Q. Why not a neural network?

The brief explicitly says prioritise explainability over unnecessarily complex deep learning, and to use neural networks only for real satellite segmentation if there is time and data. We have 3,224 samples and 9 features — a deep model would overfit and could not tell a planner why a location scored as it did.

Q. Why not Random Forest?

We could have; the accuracy would be close. Boosting fits residuals sequentially so it usually edges out bagging on structured data this size, and XGBoost's regularisation plus early-stopping controls help on an unbalanced sample. The brief names both as acceptable.

Q. What is ROC AUC and is 0.956 good?

It is the probability that the model ranks a randomly chosen developed location above a randomly chosen undeveloped one. 0.5 is a coin flip, 1.0 is perfect. 0.956 is strong. More importantly the 5-fold cross-validated figure is 0.939 ± 0.038 , so it is not a lucky split.

Q. How do you know it is not overfitting?

Three controls. A stratified 75/25 hold-out. Five-fold cross-validation that agrees with the hold-out. And explicit leakage control: the 'distance to built land' feature draws its references from a held-out half of the positives, so a parcel cannot count itself and memorise its own label.

Q. So does this predict 2030 or not?

It predicts development pressure, not a dated outcome — and we are careful about the difference. A true temporal forecast needs observed built-up extent at two dates, from GHSL or Sentinel-2. Our 2018–2026 history is modelled, so training on it would just relearn our own formula. We built the feature pipeline that a real forecast needs and left the label honest.

Engineering

Q. The brief says PostGIS is mandatory. Where is it?

We do not have it, and here is the reasoning. Every ST_* operation listed has a direct equivalent in what we use — ST_Contains is a shapely prepared predicate, ST_DWithin is an indexed raster window, ST_Distance is haversine, and our STRtree is the same R-tree PostGIS builds for GiST. At 2,567 parcels the whole city fits in memory and every query is sub-second, so a database would have added an operational dependency without changing a number. It becomes necessary the moment we scale past one city, which is exactly where we would add it.

Q. Can this scale to all of Gujarat?

The compute scales; the delivery model does not, and we have measured it. Memory is trivial — a 250 m grid over the whole state is 5.4M cells, about 43 MB per array. The rasteriser vectorises from 0.8 min to 1.9 s. What breaks is that /api/parcels returns every parcel in one response: at roughly 1 KB per parcel and about 290,000 parcels statewide, that is a 290 MB payload. The fix is PostGIS plus vector tiles and bbox-scoped queries. We would serve the state at taluka resolution and keep parcel-level work scoped to a selected district.

Q. Why MapLibre rather than Google Maps or Mapbox?

MapLibre is the open-source fork of Mapbox GL — no access token, no per-view licence cost, and it renders vector tiles with full client-side styling, which is what lets us restyle parcels by land use and drive heatmaps. Google Maps would not let us render our own parcel geometry this way, and the brief names MapLibre.

Q. How is the frontend kept in sync with four study areas?

Every API route takes `?city=`. Switching area re-points the client, re-fetches the layers and re-runs every panel's queries. The layers are fetched at runtime rather than bundled because four cities of geometry would be roughly 14 MB of JavaScript.

Q. How do you know it works?

Two end-to-end harnesses. ``npm run demo`` walks the brief's 12-step demo story through the engine and asserts the numbers hold together — the site chosen in step 8 is the one simulated in step 10 — on all four study areas. ``npm run verify:ui`` drives the real interface in a headless browser and makes 31 assertions that each panel renders engine-computed content, failing on any console error or failed request.

Awkward questions

Q. Isn't this just a dashboard with a chatbot?

No. A dashboard shows what exists. This ranks 2,567 parcels against six weighted criteria, recomputes coverage from a population raster when you place a hospital, and explains why a site won. And the assistant is not a chatbot over the data — it is a natural-language controller for the analysis engine. It cannot state a number the engine did not compute.

Q. What happens if the AI hallucinates a number?

Structurally it cannot. The LLM only selects a tool and phrases a result that has already been computed; the raw structured result is returned alongside the prose so the UI renders real values. Turn Ollama off entirely and the same questions are answered by a deterministic router with the same figures.

Q. What is the weakest part of this project?

No satellite imagery. It is the root of three limits at once: the built-up history is modelled, the growth model cannot be a dated forecast, and NDVI-type vegetation analysis is not real. One GHSL or Sentinel-2 ingest would upgrade all three, and the feature pipeline is already built for it.

11. Known weaknesses and how to answer them

Rehearse these. A specific, confident admission of a limit reads as competence.

Weakness	Say this
No satellite imagery (§31)	“Our biggest gap, and we know exactly what it blocks: dated forecasting, real vegetation indices and observed built-up change. The pipeline is built to take GHSL or Sentinel-2 — only the label changes.”
No PostGIS (§42)	“Deliberate at this scale, and here is the equivalence table. It is the first thing we add when we go past one city.”
Growth model is pressure, not a forecast	“We could have called it a 2030 prediction. We did not, because we cannot back that claim without two observed dates.”
Straight-line, not road-network distance	“The brief permits drive-time approximations for a prototype. OSRM is a day's work and would upgrade the 15-minute analysis.”
Ownership and zoning modelled	“Disclosed everywhere they are used. We would rather show a confidence band than a false certainty.”
NDVI / UHI layers are illustrative	Fix before demo. If they are still present: “those two are illustrative gradients, not satellite products — they are labelled as placeholders.”
Not deployed	“Runs locally with one command, <code>`npm run dev`</code> . Deployment is a Vercel plus Cloud Run configuration, not an architectural change.”

Weakness	Say this
No unit tests	"We have two end-to-end harnesses that assert real behaviour — 12 engine steps and 31 UI assertions. Unit tests are the next layer."

12. Glossary

Term	Meaning
GLIS	Government Land Information System — the authoritative land-records system this product is designed to sit on top of
GeoJSON	JSON format for geographic features — geometry plus properties
Parcel	A bounded piece of land. Here: an OSM land polygon, not a cadastral title plot
Ward	Municipal administrative subdivision. Ahmedabad has 48
Raster / grid	Space divided into regular cells, each holding a value. Ours is 250 m, holding population
STRtree	Sort-Tile-Recursive R-tree — a spatial index for fast "what is near here?" queries
Haversine	Great-circle distance between two lat/long points
Centroid	The geometric centre of a polygon
Buffer	A zone within a given distance of a feature
Catchment	The population a facility can plausibly serve
NDVI	Normalised Difference Vegetation Index = $(\text{NIR}-\text{RED})/(\text{NIR}+\text{RED})$, from satellite bands. Not currently computed here
NDBI / NDWI	Equivalent indices for built-up land and water. Also not computed here
GHSL	Global Human Settlement Layer — published historical built-up rasters
Sentinel-2	ESA satellite constellation providing free multispectral imagery
Overpass API	Query service for extracting OpenStreetMap data
ROC AUC	Probability the model ranks a random positive above a random negative. 0.5 = chance, 1.0 = perfect
Cross-validation	Repeatedly train on part of the data and test on the rest, to check a score is not a lucky split
Gradient boosting	Ensemble that adds trees sequentially, each correcting the previous ones' errors
Feature importance	How much each input contributed to the model's decisions
MCDA	Multi-criteria decision analysis — scoring options against several weighted criteria
15-minute city	Planning idea that essential services should be reachable in ~15 minutes
Vector tiles	Map data cut into tiles as geometry rather than images, styled on the client

Prepared for the UrbanLens team · all figures measured on commit d54406e · verify with `npm run demo`, `npm run verify:ui` and `api/ml/model`.